Linear Search

```java
//Linear Search
public class LinarSearch{
    public static void main(String[]args){
      int a[]={};
      int target=60;

     int res= search(a,target);

     if(res==1){
        System.out.println(target +" Found At: "+res);
     }else{
        System.out.println(target+ " Not Found:");
     }
    }
    static int search(int a[],int target){

        // Case 1: Array is null
        if (a == null) {
            return -1;
        }

        // Case 2: Array is empty
        if (a.length == 0) {
            return -1;
        }

         for(int i=0;i<a.length;i++){
            if(a[i]==target){
                return 1;
            }
         }
         return -1;
    }

}
```

linear_search_in_String

```
import java.util.Arrays;

public class linear_search_in_String {
    public static void main(String[]args){
      String name="Prashant";
        System.out.println("Your String is : "+name);
        char target='a';
        String res=search(name,target);
        System.out.println(res);
    }
    static String search(String name,char target){
        for(char ch:name.toCharArray()){
            if(ch==target){
                return "found";
            }
        }
        return "not Found";

    }
    }
```

Max Number in Array

```
import java.util.Arrays;

public class max_in_Array {
    public static  void main(String[]args){

        int a[]={12,45,23,78,34,545,9};
        System.out.println("Array is : "+Arrays.toString(a));
        int max=max_Element(a);
        System.out.println("Max Element in your Array is :"+max);
    }
    static int max_Element(int[]arr){

        int max=Integer.MIN_VALUE;
        for(int i=0;i<arr.length;i++){
            if(arr[i]>max){
                max=arr[i];
            }
        }
        return max;
    }
}

```

Min Number in Array

```java
import java.util.Arrays;

public class min_in_Array {
    public static void main(String[]args){
        int[] arr={10,20,12,45,23,78,34,545,9};
        System.out.println("Your Array is : "+Arrays.toString(arr));
        int min=min_Element(arr);
        System.out.println("Minimum Element in Array is : "+min);
    }
    static int min_Element(int[] arr){
        int min=Integer.MAX_VALUE;
        for (int i=0;i<arr.length;i++){
            if(arr[i]<min){
                min=arr[i];
            }
        }
        return min;
    }
}

```

search_betn_index

```
1. import java.util.Arrays;
2.
3. public class search_betn_index {
4.     public static void main(String[]args){
5.         int[] arr={10,20,12,45,23,78,34,545,9};
6.         System.out.println("Your Array is : "+Arrays.toString(arr));
7.         int start=2;
8.         int end=6;
9.         int target = 9;
10.        int res=search(arr,target,start,end);
11.        System.out.println("rsult is: "+res);
12.    }
13.    static int search(int arr[],int target, int start,int end){
14.        for(int i=start;i<=end;i++){
15.            if(arr[i]==target){
16.                return i;
17.
18.            }
19.        }
20.        return -1;
21.    }
22.
23.    }
24.
```

Swap Two Elements Array

```java
//swap the any 2 Elements in Array
import java.util.Arrays;

public class SwapArray {
    public static void main(String[]args){
        int[] a={10,20,30,40,50};
        System.out.print("Before Swap : "+Arrays.toString(a));
        swap(a,2,4);
        System.out.println("After Swap : "+Arrays.toString(a));
    }
    static void swap(int[] a,int b,int c){
        int temp=a[b];
        a[b]=a[c];
        a[c]=temp;
    }
}
```

Reachest Customer

```java
//Leetcode 1672
public class reachest_customer_wealth {
    public static void main(String[]args){
    int[][] accounts = {
    {122,3},
    {34},
    {1,3}};
    int res=find(accounts);
    System.out.println(res);
}
static int find(int accounts[][]){

int max_wealth=Integer.MIN_VALUE;
    for(int i=0;i<accounts.length;i++){
        int sum=0;
        for(int j=0;j<accounts[i].length;j++){
            sum=sum+accounts[i][j];
        }
        if(sum>max_wealth){
            max_wealth=sum;
        }
    }

    return max_wealth;
}
}

```

1. Jagged Array

```
public class jagged_array {
    public static void main(String[]args){
        int arr[][]={
            {12,45,89,34,76,43,23},
            {12,54,98,34,11},
            {111,3454,56,0}
        };
        //print jagged Array
        for(int i=0;i<arr.length;i++){
            for(int j=0;j<arr[i].length;j++){
                System.out.print(arr[i][j]+" ");
            }
            System.out.println();

        }
        //find Element in Array
        int target=0;
        int res[]= search(arr,target);
         if (res[0] != -1) {
            System.out.println("Element found at row: " + res[0]);
            System.out.println("Element found at column: " + res[1]);
        } else {
            System.out.println("Element not found");
        }
    }
    static int[] search(int[][] arr,int target){
        for(int i=0;i<arr.length;i++){
             for(int j=0;j<arr[i].length;j++){
                   if(arr[i][j]==target){
                    return new int[]{i, j};
                   }
            }
        }
        return new int[]{-1,-1};

    }
}
```

1. Even num Digit Counnt

```
1. Even num Digit Counnt
2. import java.util.Arrays;
3.
4. public class Even_num_digit {
5.     public static void main(String[]args){
6.         int[] arr={10,20,12,45,23,78,34,545,9};
7.         System.out.println("Your Array is : "+Arrays.toString(arr));
8.         int res=find_num(arr);
9.         System.out.println(res);
10.    }
11.    static int find_num(int arr[]){
12.
13.        int count=0;
14.        for (int i=0;i<arr.length;i++){
15.            if(even(arr[i])){
16.                count++;
17.            }
18.
19.        }
20.        return count;
21.    }
22.   static boolean even(int num) {
23.
24.        int digits = 0;
25.
26.        while (num > 0) {
27.            digits++;
28.            num = num / 10;
29.        }
30.
31.        return digits % 2 == 0;
32.    }
33.}
34.
35.
```

Binary Search

```
1. public class Binary_search {
2.     public static void main(String[]args){
3.         int[] arr={11,14,34,56,78,91,95,99,333};
4.         int target=99;
5.         int res= Binary_Search(arr,target);
6.
7.         System.out.println("Found at : "+res);
8.     }
9.     static int Binary_Search(int arr[],int target){
10.        int start=0;
11.        int end=arr.length-1;
12.
13.        while(start<end){
14.        int mid=(start+end)/2;
15.        if(arr[mid]==target){
16.            return mid;
17.        }else if(arr[mid]<target){
18.            start=mid+1;
19.        }else{
20.            end=mid-1;
21.        }
22.        }
23.        return -1;
24.
25.    }
26.}
27.
```

In Binary if not found then Return Greater

```java
public class Binary_greater_if_not_found {
    public static void main(String[] args) {
        int arr[] = {7, 9, 11, 23, 45, 67, 89, 90, 98};
        int target = 8;

        int res = Binary_search(arr, target);
        System.out.println(res);
    }

    static int Binary_search(int[] arr, int target) {
        int start = 0;
        int end = arr.length - 1;

        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (arr[mid] == target) {
                return arr[mid];
            } else if (arr[mid] < target) {
                start = mid + 1;
            } else {
                end = mid - 1;
            }
        }

        if (start < arr.length) {
            return arr[start];
        }

        return -1;
    }
}
```

smallest_letter_greater_than_target

```
1. public class smallest_letter_greater_than_target {
2.     public static void main(String[] args) {
3.         char[] arr={'c', 'f', 'j'};
4.         char target ='d';
5.         char ch=search(arr,target);
6.         System.out.println(ch);
7.     }
8.     static char search(char[] arr,char target){
9.
10.        int start=0;
11.        int end=arr.length-1;
12.        while(start<=end){
13.            int mid = start + (end - start) / 2;
14.            if(arr[mid]>target){
15.                end=mid-1;
16.            }else {
17.                start=mid+1;
18.            }
19.        }
20.         return arr[start%arr.length];
21.    }
22.}
23.
```

first_and_last_element_inSorted

```java
import java.util.Arrays;

public class first_and_last_element_inSorted {

    public static void main(String[] args) {
        int[] arr = {2, 3, 4, 6, 7, 7, 7, 7, 8, 9, 12, 45};
        int target = 7;

        int[] ans = search(arr, target);

        System.out.println(Arrays.toString(ans));
    }

    static int[] search(int[] arr, int target) {
        int[] ans = {-1, -1};

        ans[0] = binarySearch(arr, target, true);

        if (ans[0] != -1) {
            ans[1] = binarySearch(arr, target, false);
        }

        return ans;
    }

    static int binarySearch(int[] arr, int target, boolean findStartIndex) {
        int start = 0;
        int end = arr.length - 1;
        int ans = -1;

        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (target < arr[mid]) {
                end = mid - 1;
            } else if (target > arr[mid]) {
                start = mid + 1;
            } else {
                ans = mid;

                if (findStartIndex) {
                    end = mid - 1;
                } else {
                    start = mid + 1;
                }
```

```
            }
        }

        return ans;
    }
}
```

find_target_in_sorted_infinite_array

```
public class find_target_in_sorted_infinite_array {

    public static void main(String[] args) {
        int[] arr = {3, 5, 7, 9, 10, 90,
                     100, 130, 140, 160, 170};

        int target = 10;

        System.out.println(search(arr, target));
    }

    static int search(int[] arr, int target) {

        // First find the range
        int start = 0;
        int end = 1;

        // Expand the range until target is within it
        while (end < arr.length && target > arr[end]) {

            int temp = end + 1;

            // Double the box size
            end = end + (end - start + 1) * 2;

            // Keep end within the actual array
            if (end >= arr.length) {
                end = arr.length - 1;
            }

            start = temp;
        }

        return binarySearch(arr, target, start, end);
    }
```

```
36.     static int binarySearch(int[] arr, int target,
37.                             int start, int end) {
38.
39.         while (start <= end) {
40.             int mid = start + (end - start) / 2;
41.
42.             if (target < arr[mid]) {
43.                 end = mid - 1;
44.             } else if (target > arr[mid]) {
45.                 start = mid + 1;
46.             } else {
47.                 return mid;
48.             }
49.         }
50.
51.         return -1;
52.     }
53.}
54.
```

Search In Mountain Array

```java
public class SearchInMountainArray {

    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7, 6, 4, 2};
        int target = 4;

        SearchInMountainArray obj = new SearchInMountainArray();
        int ans = obj.search(arr, target);

        System.out.println("Target found at index: " + ans);
    }

    // Find target in a mountain array
    int search(int[] arr, int target) {

        int peak = peakIndexInMountainArray(arr);

        // Search in ascending half
        int firstTry = orderAgnosticBS(arr, target, 0, peak);

        if (firstTry != -1) {
            return firstTry;
        }

        // Search in descending half
        return orderAgnosticBS(arr, target, peak + 1, arr.length - 1);
    }

    // Find the peak index
    int peakIndexInMountainArray(int[] arr) {

        int start = 0;
        int end = arr.length - 1;

        while (start < end) {

            int mid = start + (end - start) / 2;

            if (arr[mid] > arr[mid + 1]) {
                end = mid;
            } else {
                start = mid + 1;
            }
        }

```

```
46.         return start;
47.     }
48.
49.     // Binary search for ascending or descending arrays
50.     static int orderAgnosticBS(
51.             int[] arr, int target, int start, int end) {
52.
53.         boolean isAsc = arr[start] < arr[end];
54.
55.         while (start <= end) {
56.
57.             int mid = start + (end - start) / 2;
58.
59.             if (arr[mid] == target) {
60.                 return mid;
61.             }
62.
63.             if (isAsc) {
64.                 if (target < arr[mid]) {
65.                     end = mid - 1;
66.                 } else {
67.                     start = mid + 1;
68.                 }
69.             } else {
70.                 if (target > arr[mid]) {
71.                     end = mid - 1;
72.                 } else {
73.                     start = mid + 1;
74.                 }
75.             }
76.         }
77.
78.         return -1;
79.     }
80. }
81.
```

Split Array

```java
public class SplitArray {

    public static void main(String[] args) {
        int[] nums = {7, 2, 5, 10, 8};
        int m = 2;
        SplitArray obj = new SplitArray();
        int ans = obj.splitArray(nums, m);

        System.out.println("Minimum largest sum: " + ans);
    }
    // https://leetcode.com/problems/split-array-largest-sum/
    public int splitArray(int[] nums, int m) {
        int start = 0;
        int end = 0;

        for (int num : nums) {
            start = Math.max(start, num);
            end += num;
        }
        // Binary search for the minimum possible largest sum
        while (start < end) {

            int mid = start + (end - start) / 2;
            int sum = 0;
            int pieces = 1;

            for (int num : nums) {

                if (sum + num > mid) {
                    sum = num;
                    pieces++;
                } else {
                    sum += num;
                }
            }
            if (pieces > m) {
                start = mid + 1;
            } else {
                end = mid;
            }
        }

        return end;
    }
}
```

RotationCount

```java
public class RotationCount {
    public static void main(String[] args) {
        int[] arr = {4,5,6,7,0,1,2};
        System.out.println(countRotations(arr));
    }
    private static int countRotations(int[] arr) {
        int pivot = findPivot(arr);                          // only this for used to Calculate Count
        return pivot + 1;
    }
    // use this for non duplicates
    static int findPivot(int[] arr) {
        int start = 0;
        int end = arr.length - 1;
        while (start <= end) {
            int mid = start + (end - start) / 2;
            // 4 cases over here
            if (mid < end && arr[mid] > arr[mid + 1]) {
                return mid;
            }
            if (mid > start && arr[mid] < arr[mid - 1]) {
                return mid-1;
            }
            if (arr[mid] <= arr[start]) {
                end = mid - 1;
            } else {
                start = mid + 1;
            }
        }
        return -1;
    }

    // use this when arr contains duplicates
    static int findPivotWithDuplicates(int[] arr) {
        int start = 0;
        int end = arr.length - 1;
        while (start <= end) {
            int mid = start + (end - start) / 2;
            // 4 cases over here
            if (mid < end && arr[mid] > arr[mid + 1]) {
                return mid;
            }
```

```
42.             if (mid > start && arr[mid] < arr[mid - 1]) {
43.                 return mid-1;
44.             }
45.
46.             // if elements at middle, start, end are equal then just skip
    the duplicates
47.             if (arr[mid] == arr[start] && arr[mid] == arr[end]) {
48.                 // skip the duplicates
49.                 // NOTE: what if these elements at start and end were the
    pivot??
50.                 // check if start is pivot
51.                 if (arr[start] > arr[start + 1]) {
52.                     return start;
53.                 }
54.                 start++;
55.
56.                 // check whether end is pivot
57.                 if (arr[end] < arr[end - 1]) {
58.                     return end - 1;
59.                 }
60.                 end--;
61.             }
62.             // left side is sorted, so pivot should be in right
63.             else if(arr[start] < arr[mid] || (arr[start] == arr[mid] &&
    arr[mid] > arr[end])) {
64.                 start = mid + 1;
65.             } else {
66.                 end = mid - 1;
67.             }
68.         }
69.         return -1;
70.     }
71. }
72.
```